

Pre-Entrega Módulo 3

Capa de Persistencia Correlacionada y Memoria

Memoria de Largo Plazo con Airtable y Summarization Agéntica en n8n

Lautaro Bustos Raiz

Proyecto Integrador — Trayecto de Automatización con Agentes de IA

1. Introducción y Objetivo

Este documento describe la evolución de la arquitectura Manager–Worker del Módulo 2 hacia una capa de persistencia correlacionada y memoria. El sistema mantiene los tres flujos independientes en n8n (Manager, Worker 1 y Worker 2), e incorpora un circuito de memoria de largo plazo respaldado en Airtable: lectura del historial por Session_ID, inyección de contexto en el System Prompt del AI Agent Router, y escritura idempotente (Create/Update) que incluye una Summarization automática cuando la sesión supera los 5 intercambios. Worker 1 además incorpora una lógica anti-reasignación de técnico para casos reabiertos.

Las correcciones solicitadas sobre la entrega del Módulo 2 (consistencia de acentuación, ejemplos JSON completos, capturas ampliadas de configuración, tabla de pruebas de regresión y revisión de expresiones antes de exportar) se aplican también en esta entrega y se detallan en las secciones 9 a 11.

2. Arquitectura General — Workflow Manager M3

El Manager del Módulo 3 extiende la arquitectura del M2 incorporando, antes del AI Agent Router, un circuito de lectura de memoria: el nodo Airtable Search ("Leer Memoria Airtable") recupera el historial de la sesión filtrando por Session_ID, y el nodo IF "¿Tiene historial previo?" bifurca el flujo entre usuario nuevo ("Set Contexto (nuevo)") y usuario recurrente ("Set Contexto (recurrente)"). Ambas ramas conforman el mismo shape de datos (chatInput, session_id, tiene_memoria, mensaje_count, airtable_record_id, contexto_previo, estado_previo) antes de llegar al AI Agent Router, que inyecta ese contexto en su System Prompt mediante los delimitadores [INICIO DE CONTEXTO COMPARTIDO] / [FIN DEL CONTEXTO COMPARTIDO].

Después de la clasificación y el enrutamiento (idéntico al M2: TECH_SUPPORT → Worker 1, BILLING → Worker 2, fallback → ESCALATE_HUMAN), un segundo circuito de persistencia evalúa con el IF "¿Más de 5 mensajes?" si corresponde activar el Summarization Agent. El resultado se persiste de forma idempotente: Update con resumen comprimido (si hubo Summarization), Update simple (sesión recurrente sin Summarization) o Create (sesión nueva).

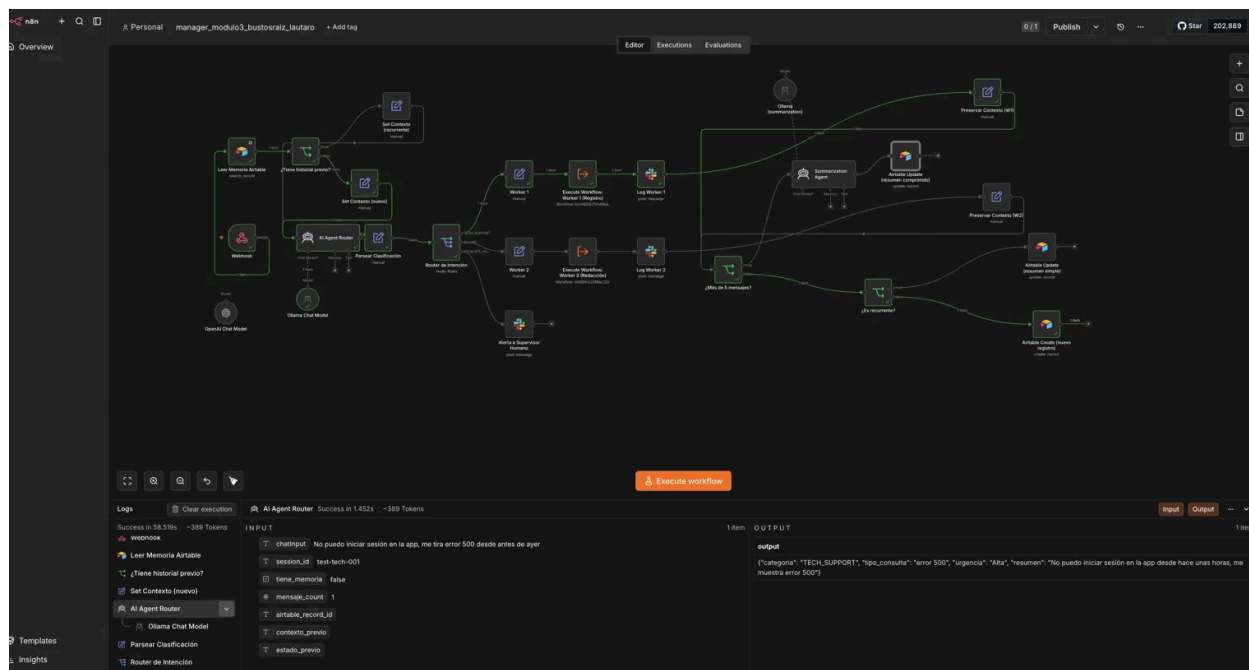


Figura 1 — Ejecución real del Manager M3 (session_id: test-tech-001): el AI Agent Router clasifica "No puedo iniciar sesión en la app, me tira error 500 desde antes de ayer" como categoría: TECH_SUPPORT, urgencia: Alta, con tiene_memoria: false (usuario nuevo).

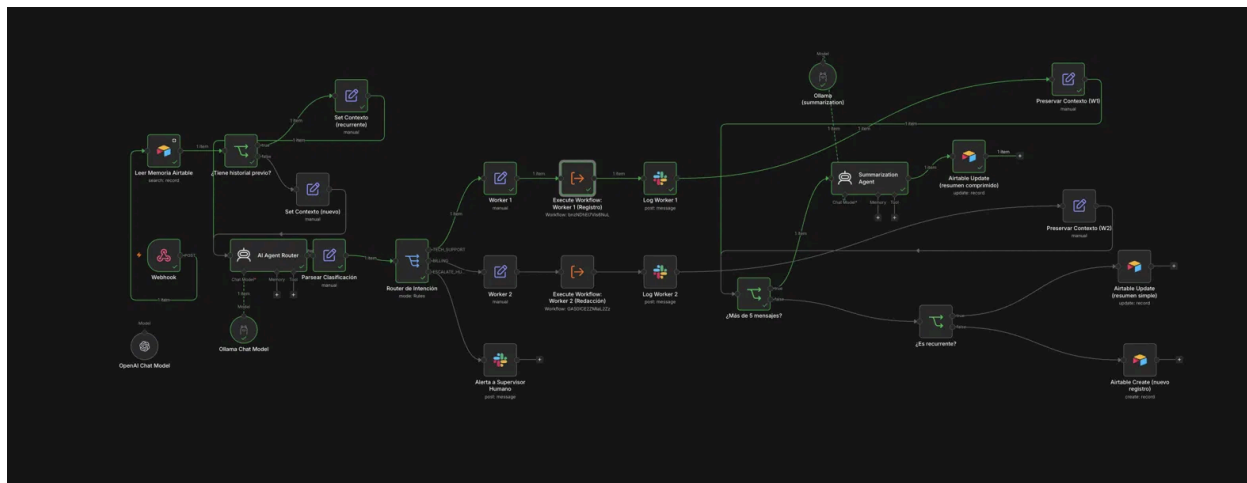


Figura 2 — Canvas completo del Manager M3: circuito de memoria (izquierda), AI Agent Router y Switch (centro), Workers y rama de Summarization con Airtable Update/Create (derecha).

3. Circuito de Memoria Persistente

3.1 Esquema de datos — tabla Memoria_Agente (Airtable)

La tabla actúa como base de datos de largo plazo, indexada por Session_ID, garantizando el aislamiento de contexto entre sesiones distintas.

Campo	Tipo en Airtable	Descripción	Operación
Session_ID	Single line text	Identificador único de sesión (viene del body del webhook)	Llave de búsqueda/filtro
Fecha_Actualizacion	Date (con hora)	Timestamp del último registro o actualización	Escritura en Create/Update
Resumen_Consolidado	Long text	Resumen JSON comprimido generado por el Summarization Agent, o texto de la primera consulta	Lectura para inyectar en prompt / Escritura al cerrar sesión
Estado_Caso	Single line text	abierto cerrado escalado — extraído del JSON del Summarization Agent	Escritura en Update
Datos_Clave	Long text	Array JSON de puntos clave extraídos por el Summarization Agent, o chatInput inicial	Escritura en Update
Mensaje_Count	Number	Contador de intercambios en la sesión. Al superar 5, activa la Summarization	Lectura para IF / incremento en cada escritura
Tecnico_Asignado	Single line text	Nombre del técnico de Postgres asignado al caso. Se persiste en el primer Create para evitar reasignaciones	Lectura para detectar caso reabierto / Escritura en primer Create

3.2 Circuito de lectura → inyección → escritura

El flujo de persistencia sigue el patrón:

- (1) Search por Session_ID en Airtable.
- (2) IF historial previo ("¿Tiene historial previo?").

- (3) Inyección en el System Prompt con delimitadores [INICIO DE CONTEXTO COMPARTIDO] / [FIN DEL CONTEXTO COMPARTIDO].
- (4) Ejecución del AI Agent Router y del Worker correspondiente.
- (5) IF mensaje_count >= 5 ("¿Más de 5 mensajes?").
- (6a) TRUE → Summarization Agent + Airtable Update (resumen comprimido) idempotente.
- (6b) FALSE + recurrente → Airtable Update (resumen simple).
- (6c) FALSE + nuevo → Airtable Create (nuevo registro).

3.3 Prompt y ejemplo del Summarization Agent

El Summarization Agent se activa cuando el contador de mensajes de la sesión supera 5 intercambios. Recibe la consulta actual del usuario junto con el contexto previo recuperado de Airtable.

System Message del Summarization Agent

Sos un sistema de compresión de memoria para un agente de soporte.
Respondé ÚNICAMENTE con un objeto JSON válido, sin texto adicional ni markdown, con este formato exacto:

```
{
  "asunto_principal": "descripción en una oración del problema central",
  "puntos_clave": ["punto 1", "punto 2"],
  "accion_requerida": "próximo paso concreto",
  "estado": "abierto|cerrado|escalado"
}
```

Campo Text (input del agente)

```
={{ 'Consulta del usuario: ' + $json.chatInput +
  '\n\nContexto previo: ' + ($json.contexto_previo || 'Sin historial previo') +
  '\n\nTécnico asignado previamente: ' + ($json.tecnico_asignado_previo || 'Ninguno') }}
```

Ejemplo de salida real generada (capturada en ejecución)

```
{
  "asunto_principal": "Error 500 al iniciar sesion en la aplicacion",
  "puntos_clave": ["Error 500 desde antes de ayer", "Iniciar sesion falla"],
  "accion_requerida": "Contactar a Martina Gomez para resolver el problema",
  "estado": "abierto"
}
```

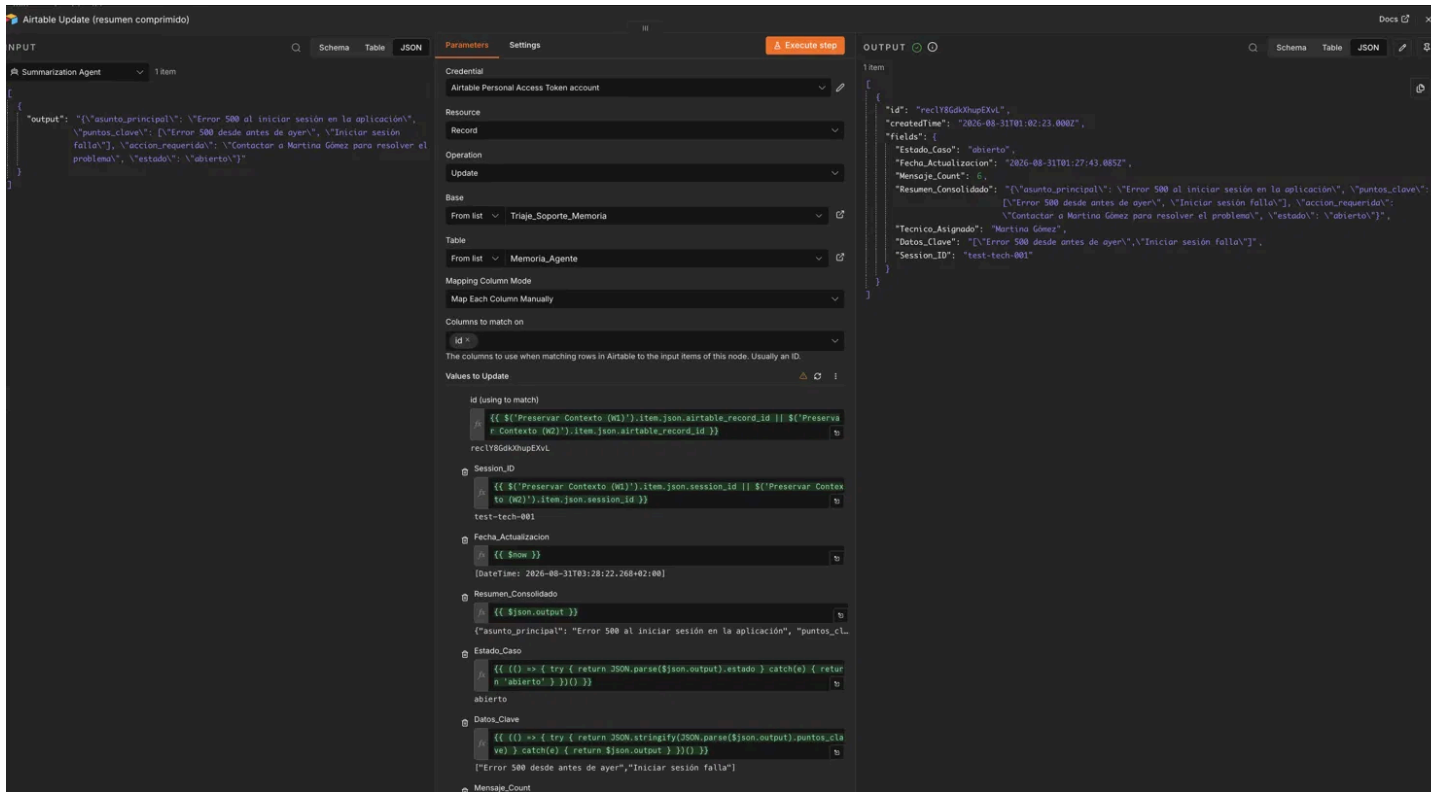


Figura 3 — Nodo Airtable Update (resumen comprimido) ampliado: Valores to Update completo, con el output real del Summarization Agent mapeado a Resumen_Consolidado, Estado_Caso y Datos_Clave.

Evidencia de escritura idempotente — antes y después del Summarization

La misma sesión (test-tech-001) antes de superar los 5 intercambios, con Resumen_Consolidado igual al chatInput inicial:

Triaje_Soporte_Memoria									
Memoria_Agente									
	Session_ID	Fecha_Actualizacion	Resumen_Consolidado	Estado_Caso	Datos_Clave	Mensaje_Count	Tecnico_Asignado		
1	test-tech-001	30/8/2026 10:02pm	No puedo iniciar sesi\u00f3n en la app, me tira error 500 desde antes de ayer	abierto	No puedo iniciar sesi\u00f3n e...	1.0	Martina G\u00f3mez		

Figura 4 — Tabla Memoria_Agente antes de la Summarization: Mensaje_Count = 1, Resumen_Consolidado = texto plano de la primera consulta.

La misma fila después de que el Summarization Agent comprimi\u00f3 el historial (mismo record, actualizado v\u00eda Update por id, no duplicado):

	Session_ID	Fecha_Actualizacion	Resumen_Consolidado	Estado_Caso	Datos_Clave	Mensaje_Count	Tecnico_Asignado
1	test-tech-001	30/8/2026 10:27pm	{"asunto_principal": "Error 500 al iniciar sesión en la aplicación", "puntos_clav..."}	abierto	{"Error 500 desde antes ...	1.0	Martina Gómez

Figura 5 — Tabla Memoria_Agente después de la Summarization: Resumen_Consolidado ahora contiene el JSON comprimido (asunto_principal, puntos_clave, accion_requerida, estado).

4. Worker 1 — Registro, Búsqueda, Asignación y Anti-reasignación de Técnico

Especialista atómico que: (1) recibe tecnico_asignado_previo desde el Manager; (2) si ese campo no está vacío, deriva a la rama "caso reabierto" en vez de consultar Postgres, dejando una nota de seguimiento explícita; (3) si está vacío, consulta Postgres con ILIKE, asigna el técnico vía UPDATE con RETURNING, y registra el caso en Google Sheets; (4) en caso de error, alerta a Slack en paralelo al registro.

Ejemplo JSON de entrada (Manager → Worker 1, caso nuevo) — capturado en ejecución real

```
{
  "mensaje": "No puedo iniciar sesión en la app, me tira error 500 desde antes de ayer",
  "tipo_consulta": "bug",
  "urgencia": "Alta",
  "resumen": "No puedo iniciar sesión en la app desde hace unas horas, me muestra error 500",
  "session_id": "test-tech-001",
  "tecnico_asignado_previo": ""
}
```

Ejemplo JSON de salida — técnico asignado (Contrato de Salida éxito) — capturado en ejecución real

```
{
  "status": "success",
  "data": {
    "session_id": "test-tech-001",
    "row_appended": true,
    "fecha": "2026-08-31T03:02:22.088+02:00",
    "status_caso": "asignado",
    "tecnico_asignado": "Martina Gómez"
  }
}
```

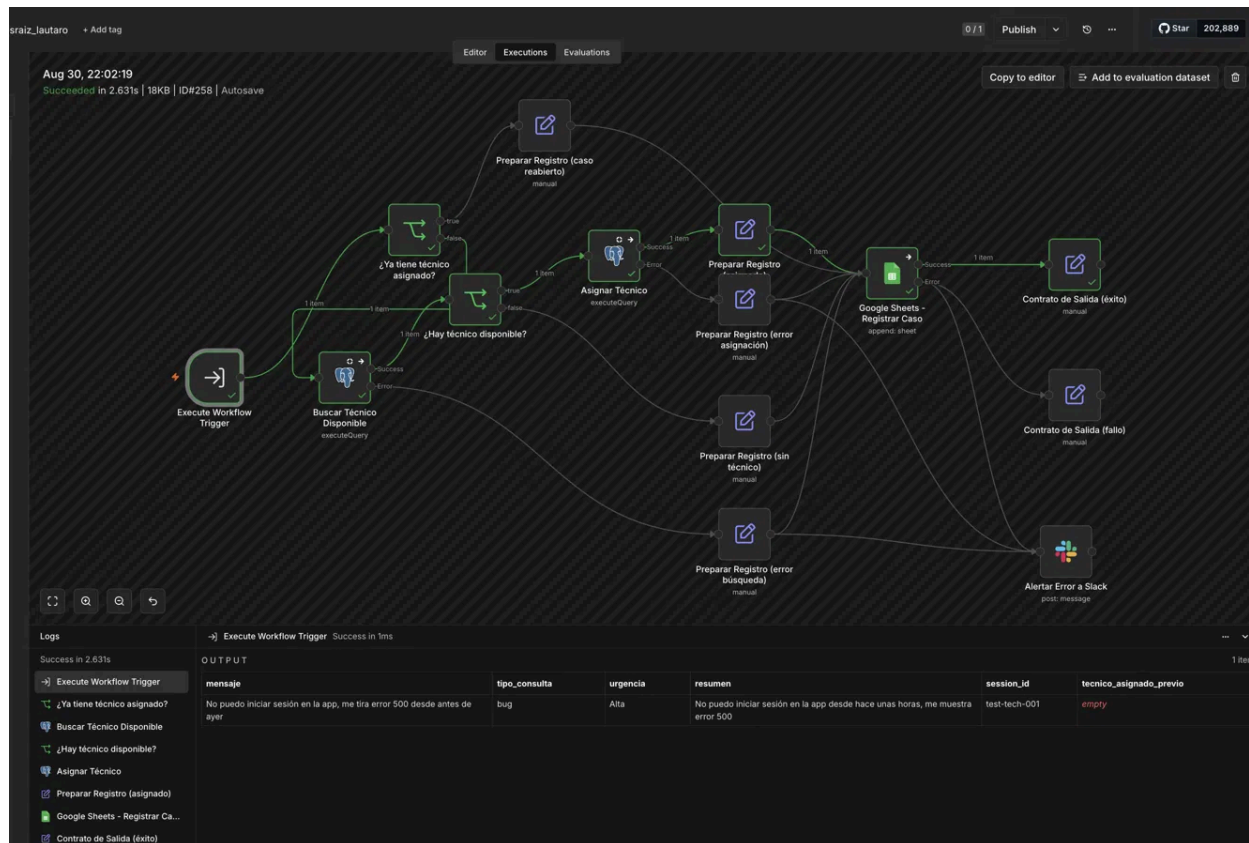


Figura 6 — Worker 1: ejecución completa en verde (ID#258). El nodo Buscar Técnico Disponible consulta Postgres, Asignar Técnico marca a Martina Gómez como ocupada, y Google Sheets - Registrar Caso deja el registro con status: asignado.

id [PK] integer	nombre character varying (100)	especialidad character varying (50)	disponible boolean	asignado_a_session character varying (100)	creado_en timestamp without time zone
2	Facundo Rivas	bug	true	[null]	2026-08-17 20:25:46.222
4	Nicolás Pereyra	reclamo	true	[null]	2026-08-17 20:25:46.222
5	Camila Duarte	reembolso	true	[null]	2026-08-17 20:25:46.222
3	Sofía Herrera	duda de uso	true	[null]	2026-08-17 20:25:46.222
1	Martina Gómez	bug	false	test-tech-001	2026-08-17 20:25:46.222
6	Tomás Aguirre	bug	true	[null]	2026-08-17 20:25:46.222

Figura 7 — Tabla tecnicos en Postgres tras la ejecución: Martina Gómez (id 1, especialidad bug) queda con disponible = false y asignado_a_session = test-tech-001.

Ejemplo JSON de entrada — caso reabierto (sesión recurrente) — ejemplo ilustrativo, pendiente de captura real

```
{
  "mensaje": "Sigo sin poder entrar, el error 500 volvió a aparecer.",
  "tipo_consulta": "bug",
  "urgencia": "Alta",
  "resumen": "El error 500 reapareció tras el primer contacto.",
  "session_id": "test-tech-001",
  "tecnico_asignado_previo": "Martina Gómez"
}
```

Ejemplo JSON de salida — caso reabierto

```
{
  "status": "success",
  "data": {
    "session_id": "test-tech-001",
  }
}
```

```
{
  "row_appended": true,
  "fecha": "2026-08-18T14:05:00.000Z",
  "status_caso": "caso_reabierto",
  "tecnico_asignado": "Martina Gómez"
}
```

Nota: la Figura 6 (sección 4) ya muestra el nodo ¿Ya tiene técnico asignado? dentro del canvas completo de Worker 1. Falta capturar una ejecución real de la rama TRUE (caso reabierto), marcada como pendiente en la sección 12.

5. Worker 2 — Redacción de Respuesta al Cliente

Especialista atómico que redacta un mensaje de respuesta orientado al cliente según la categoría de negocio recibida (TECH_SUPPORT o BILLING).

Ejemplo JSON de entrada (Manager → Worker 2)

```
{
  "mensaje": "Me cobraron dos veces el mismo plan este mes.",
  "categoria": "BILLING",
  "session_id": "71"
}
```

Ejemplo JSON de salida (Contrato de Salida éxito)

```
{
  "status": "success",
  "data": {
    "session_id": "71",
    "respuesta_sugerida": "Hola, recibimos tu reclamo de facturación. Un especialista de billing va a revisar tu caso y te contacta en menos de 24hs."
  }
}
```

6. Contrato de Datos

Cada delegación del Manager pasa por un nodo Set que transmite únicamente el mínimo producto viable, evitando el antipatrón de Data Stuffing.

Manager → Worker 1

Campo	Tipo	Origen	Descripción
mensaje	string	Webhook.body.chatInput	Texto original del cliente
tipo_consulta	string	AI Agent Router (JSON)	bug duda de uso reclamo reembolso
urgencia	string	AI Agent Router (JSON)	Baja Media Alta Crítica
resumen	string	AI Agent Router (JSON)	Resumen en una oración
session_id	string	Webhook.body.session_id	ID de sesión del usuario, persistente entre ejecuciones
tecnico_asignado_previo	string	Preservar Contexto (W1)	Vacío si es caso nuevo; nombre del técnico si la sesión es recurrente

Manager → Worker 2

Campo	Tipo	Origen	Descripción
mensaje	string	Webhook.body.chatInput	Texto original del cliente
categoria	string	AI Agent Router (JSON)	TECH_SUPPORT BILLING
session_id	string	Webhook.body.session_id	ID de sesión del usuario

Campos en Google Sheets (Worker 1)

Columna	Posibles valores	Descripción
fecha	timestamp	Hora real de registro
tipo_consulta	bug duda de uso reclamo reembolso	Clasificación del AI Router
urgencia	Baja Media Alta Crítica	Nivel de urgencia
resumen	texto libre	Resumen generado por el Router
escalar_humano	true false	true si urgencia = "Crítica"
status	asignado sin_tecnico error_asignacion error_busqueda caso_reabierto	Resultado del proceso de asignación
tecnico_asignado	nombre o vacío	Técnico asignado por Postgres, o el previo si el caso se reabrió
nota_seguimiento	texto libre o vacío	Nota de seguimiento cuando el caso es reabierto

7. Lógica de Enrutamiento y Taxonomía Cerrada

El AI Agent Router clasifica dentro de 3 categorías de negocio. El Switch evalúa el campo categoria de forma determinista (comparación exacta de string). La rama de fallback actúa como vía de escape hacia un supervisor humano.

Categoría	Condición de negocio	Ruta en el Manager
TECH_SUPPORT	Bugs, errores técnicos, problemas de acceso	→ Worker 1 (Registro + Asignación de Técnico)
BILLING	Reclamos de pago, reembolsos, facturación	→ Worker 2 (Redacción de Respuesta)
ESCALATE_HUMAN	Casos ambiguos, críticos, pérdida de datos/dinero	→ Alerta directa a Slack (bypass de Workers)

Corrección aplicada respecto al Módulo 2: se verificó que el valor "Crítica" (con tilde) se usa de forma consistente en el System Message del AI Agent Router y en la expresión escalar_humano de Google Sheets del Worker 1. No se detectaron variantes sin tilde ("Critica") en los tres workflows exportados.

8. Sincronización Manager–Worker

Ambos nodos Execute Workflow (Worker 1 y Worker 2) tienen Wait for Sub-Workflow Completion = true, pausando el Manager hasta que el Worker retorne su JSON estructurado. Esto garantiza que el log de trazabilidad en Slack reciba siempre la respuesta real del especialista.

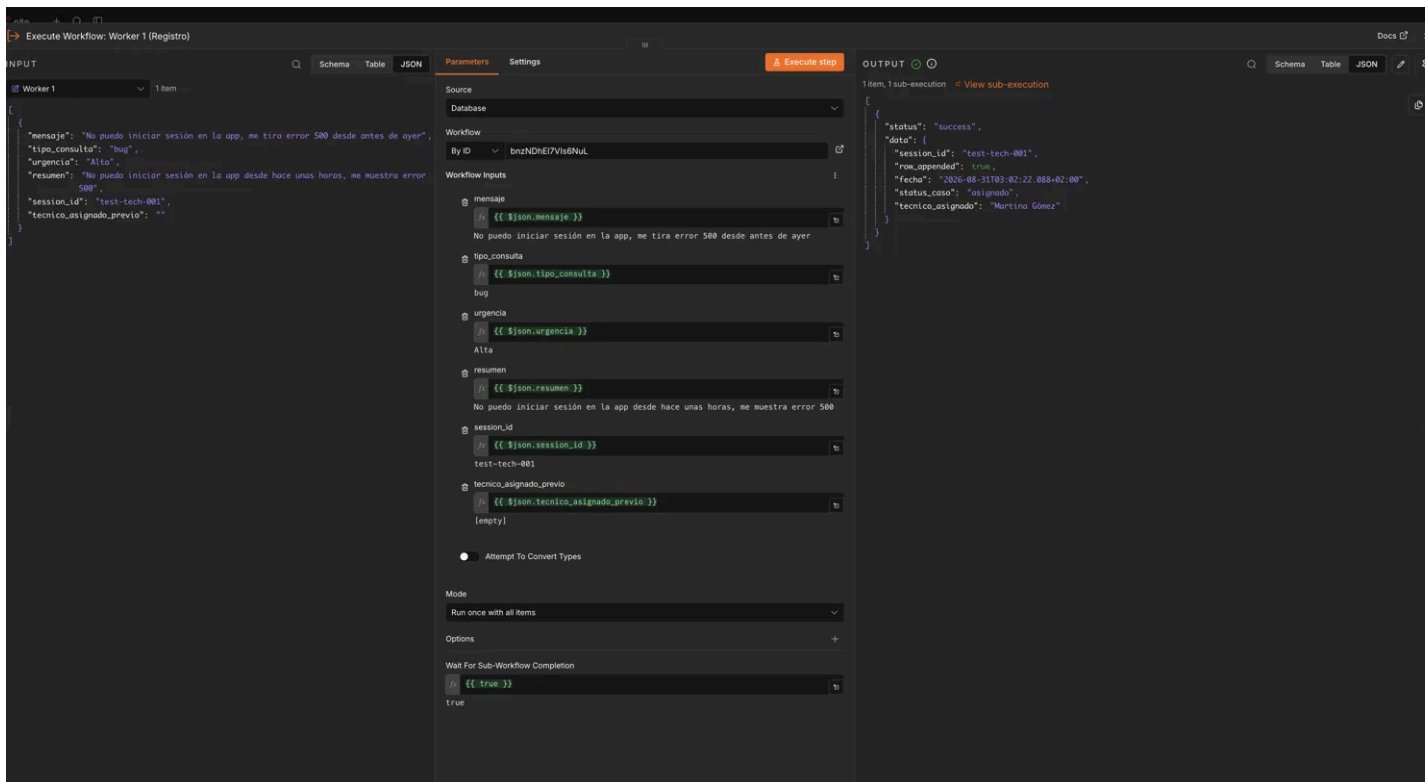


Figura 8 — Execute Workflow: Worker 1 (Registro), configuración ampliada: mapeo de inputs (defineBelow, 6 campos) y, en la parte inferior, Wait For Sub-Workflow Completion = {{ true }}. A la derecha, el output real de la sub-ejecución.

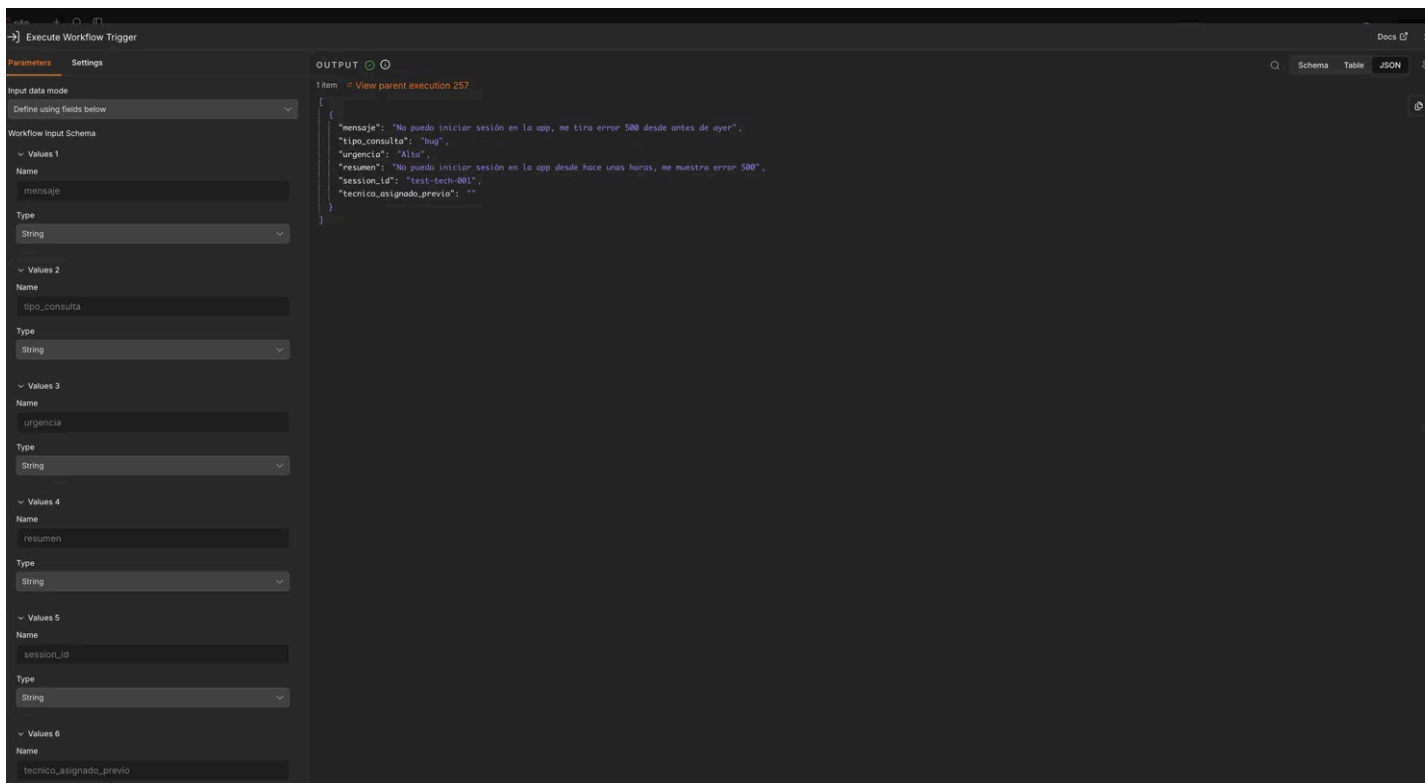


Figura 9 — Execute Workflow Trigger de Worker 1 (pestaña Settings): Workflow Input Schema completo con los 6 campos (mensaje, tipo_consulta, urgencia, resumen, session_id, tecnico_asignado_previo) y el output real recibido.

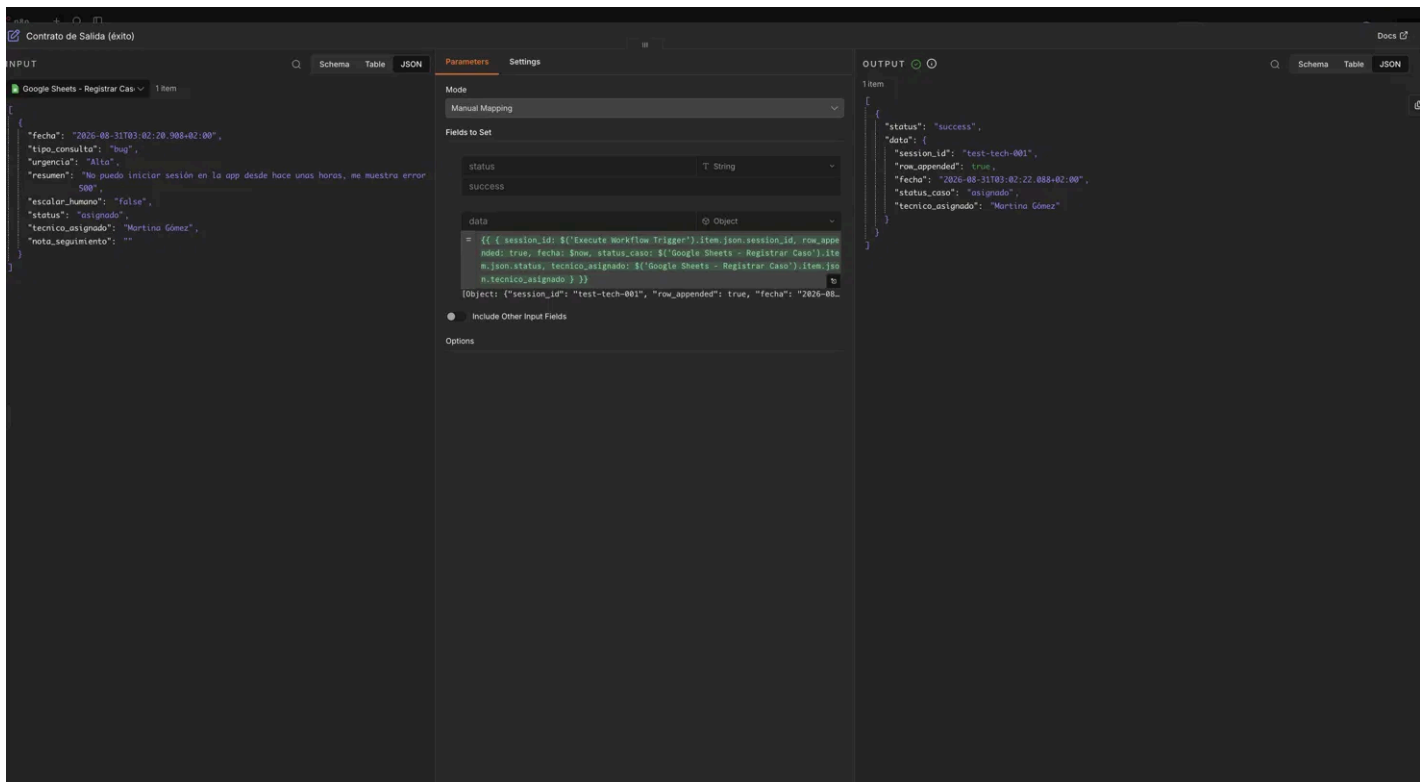


Figura 10 — Nodo Contrato de Salida (éxito) de Worker 1: expresión completa del campo data, sin recortar, junto con el output evaluado.

9. Manejo de Fallos y Casos Reabiertos

Cuando la query de Postgres retorna 0 filas, el nodo tiene activado Always Output Data para no cortar la cadena. Las rutas posibles (asignado, sin_tecnico, error_asignacion, error_busqueda, caso_reabierto) convergen en un único nodo de Google Sheets, asegurando que siempre quede un registro. Las rutas de error además disparan una alerta inmediata a Slack en paralelo.

La rama de anti-reasignación ("¿Ya tiene técnico asignado?") se evalúa antes de tocar Postgres: si tecnico_asignado_previo llega no vacío desde el Manager, el Worker no vuelve a buscar disponibilidad y conserva el técnico original, dejando trazabilidad del reclamo repetido en nota_seguimiento.

10. Observabilidad

Cada rama del Manager finaliza en un nodo Slack con: Worker invocado, parámetros enviados y respuesta recibida. Las rutas de error del Worker 1 tienen además un nodo Slack propio que se dispara en paralelo al registro en Sheets, garantizando visibilidad inmediata ante fallos sin esperar al log del Manager.

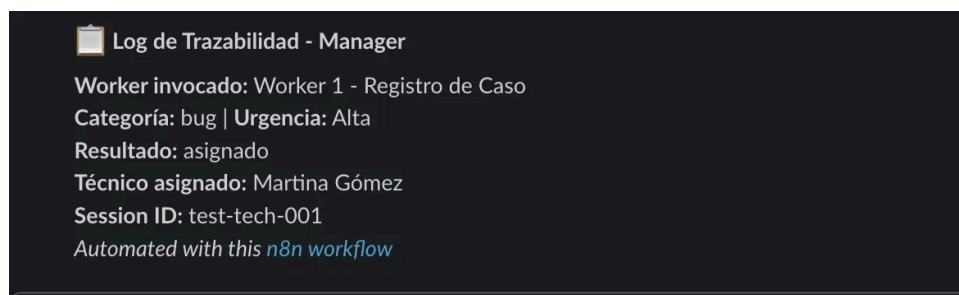


Figura 11 — Mensaje real posteoado en Slack por el nodo Log Worker 1: Worker invocado, categoría, urgencia, resultado, técnico asignado y session_id.

11. JSONs de los Workflows

Los jsons de la entrega estan en el siguiente github <https://github.com/lautapa96/curso-n8n/tree/main/Entrega3>